

In this program the password is not stored as a single string; instead, the line of code:

```
'char pass[]={ 'h', 'e', 'l', 'l', 'o', '\0' };'
```

...declares the password as an array of characters.

```
#include<stdio.h>
#include<string.h>
int main()
{
    char pas[100];
    char pass[]={ 'h', 'e', 'l', 'l', 'o', '\0' };
    printf("\nEnter the password:");
    scanf("%s", pas);
    if(!strcmp(pass, pas))
    {
        printf("\nYou have CRACKED me!!!\n");
    }
    else
    {
        printf("\nSorry\n");
    }
    return(0);
}
```

Compile this C program with the command `'gcc crack2.c -o test2'` to obtain the executable file *test2*. Now let us search this executable file with the command *strings* and the utility *objdump* to find the password. Figure 6 shows a part of the output of the command `'strings test2'` and Figure 7 shows a part of the output of the

```
File Edit View Search Terminal Help
%R
%J
%b
=9
AWAVI
AUATL
[]A\A]A^A_
Enter the password:
You have CRACKED me!!!
Sorry
;*3$"
```

Figure 6: Output of the *strings* command on *test2*

command `'objdump -s test2'`.

From Figures 6 and 7, we can clearly see that both the *strings* command and the *objdump* utility fail while trying to break the C program *crack2.c*. So, in order to attack this program, we are going to use the powerful debugger GDB (GNU Debugger) that works for many programming languages like Ada, C, C++, Objective-C, Free Pascal, FORTRAN, Go, etc. But how can we make sure that the executable file *test2* is indeed obtained from one of the above languages? A utility called *ldd*, which prints the shared libraries used by a program, comes to our help in this situation. Figure 8 shows the output of the command `'ldd test2'`.

From Figure 8, we can see that the executable file *test2* calls the file *libc.so.6*. Since the term 'libc' is often used as a short hand for the 'standard C library', we can conclude that the executable file *test2* is an executable compiled from a C program. Thus, we can readily use GDB to disassemble the executable file *test2*. The command `'gdb test2'` opens the executable file *test2* for processing. Run the program once with the command `'run'` on the prompt (gdb), before adding any breakpoints. Now execute the command `'disassemble main'`. Figure 9 shows a part of the disassembled code of

the executable file *test2*.

The line of code:

```
0x0000555555554810 <+102>: callq
0x555555554670 <strcmp@plt>
```

...in Figure 9 is the only place where the string comparison function *strcmp()* of C is used. Thus, it is a good idea to search for the password in registers used immediately before this line of code. The lines of code:

```
0x000055555555480a <+96> mov
%rdx,%rsi
and 0x000055555555480d <+99>: mov
%rax,%rdi
```

...shown in Figure 9, tell us that we have to probe the registers *rdx* and *rax* to retrieve the password. Now we set two breakpoints. The first at the beginning of the *main()* function and the second in the line of code where the function *strcmp()* is called, with the commands *b main* and *b * 0x0000555555554810*. Now run the executable file *test2* with the command *run*. The execution temporarily stops at the first breakpoint. Enter the command *c* to continue the execution and enter the test password *dummy* when prompted by the program.

```
File Edit View Search Terminal Help
0880 ff4885ed 742031db 0f1f8400 00000000 .H..t 1.....
0890 4c89fa4c 89f64489 ef41ff14 dc4883c3 L..L..D..A...H..
08a0 014839dd 75ea4883 c4085b5d 415c415d .H9.u.H...[]A\A]
08b0 415e415f c390662e 0f1f8400 00000000 A^A_...f.....
08c0 f3c3 ..
Contents of section .fini:
08c4 4883ec08 4883c408 c3 H...H....
Contents of section .rodata:
08d0 01000200 0a456e74 65722074 68652070 .....Enter the p
08e0 61737377 6f72643a 20002573 000a596f assword: .%s..Yo
08f0 75206861 76652043 5241434b 4544206d u have CRACKED m
0900 65212121 000a536f 72727900 e!!!..Sorry.
```

Figure 7: Output of *objdump* utility on *test2*

```
File Edit View Search Terminal Help
# ldd test2
linux-vdso.so.1 (0x00007ffc0f5dc000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f9a8cdf2000)
/lib64/ld-linux-x86-64.so.2 (0x00007f9a8d3e5000)
```

Figure 8: Output of the command *ldd*